TUWAIQ ACADEMY

Machine Learning

Large-Scale GPU-Accelerated Food Demand Forecasting

Supply Chain Planning with the Walmart M5 Dataset

Final Project Report

Information	Details
Student Name	Abdulrahman Abdullah Alqurashi
Instructor	Banan Alnemri
Course	Scalable Data Science
Program	AI Diploma
Submission Date	13 August 2026

Project at a glance

27,489,810 product-store-day observations were processed with RAPIDS. The final tuned Prophet forecast achieved 10.30% WMAPE and 89.70% weighted forecast accuracy on an unseen 28-day test period.

1. Abstract

This project develops a scalable forecasting workflow for retail food demand using the public Walmart M5 dataset. The business problem is to estimate future units sold so that inventory replenishment, staffing, and supply planning can be aligned with expected demand. Three source tables - daily sales, calendar information, and weekly prices - were filtered to the FOODS category and transformed with RAPIDS into 27,489,810 product-store-day observations. cuDF and CuPy were used for loading, reshaping, joining, cleaning, feature engineering, metric calculation, and GPU memory management. The detailed modeling stage compared historical baselines, three cuML regression algorithms, and two GPU XGBoost objectives on a chronological validation period. XGBoost with absolute-error loss achieved the strongest detailed result, with a test WMAPE of 57.60%, but the product series remained highly intermittent because 62.02% of observations contained zero sales. For the final business forecast, demand was therefore aggregated into 30 daily store-department series and evaluated with a four-week weekday baseline, Original Prophet, and Tuned Prophet. The tuned configuration achieved a final test WMAPE of 10.30%, weighted forecast accuracy of 89.70%, and R-squared of 0.9679. The results show that large-scale GPU processing can support detailed experimentation, while carefully chosen aggregation and chronological validation produce a practical and stable 28-day retail forecast.

Executive Summary

- Commercial use: predict food demand to support purchasing, inventory allocation, staffing, and supply planning.
- Scale: 27.49 million detailed observations derived from 1,437 products, 10 stores, 3 departments, and 1,913 sales days.
- Scalable implementation: RAPIDS cuDF and CuPy handled the large table; cuML supplied three GPU regression algorithms; NVML captured utilization and memory metrics.
- Detailed-stage winner: GPU XGBoost L1 produced the lowest product-level validation WMAPE and generalized to the final detailed test period.
- Final operational model: Tuned Prophet forecast 30 store-department series and slightly but consistently outperformed the strong weekday baseline on unseen data.
- Final result: 96.99 MAE, 146.79 RMSE, 0.9679 R-squared, 10.30% WMAPE, and 89.70% weighted forecast accuracy.

Contents

Section	Focus
1	Abstract and Executive Summary
2	Introduction
3	Problem Statement
4	Objectives
5	Data
6	Methodology and Training
7	Evaluation and Results
8	Challenges
9	Future Work
10	Discussion
11	Conclusion
12	References
Appendices	Features, hyperparameters, outputs, and reproducibility notes

Project Workflow

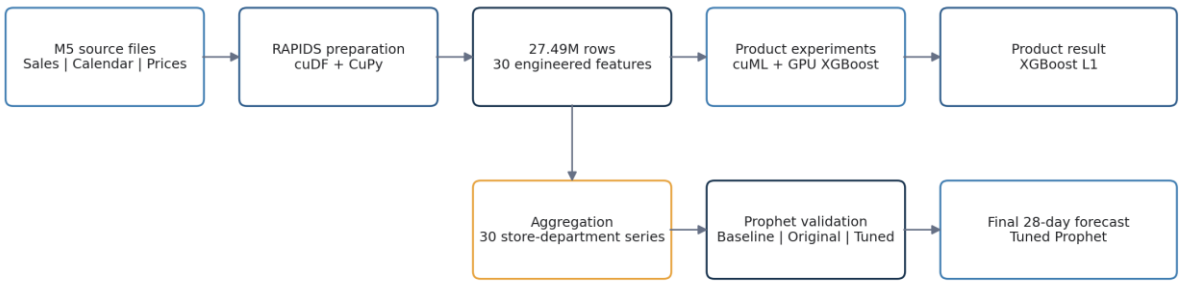


Figure 1. End-to-end workflow implemented in the notebook.

2. Introduction

Retail demand forecasting connects historical sales data to forward-looking operational decisions. A forecast that is too low can create stockouts, lost revenue, and poor customer experience. A forecast that is too high can increase holding costs, waste, markdowns, and working-capital requirements. Food retail is especially challenging because demand changes across products, stores, weekdays, seasons, events, prices, and benefit-payment schedules.

The Walmart M5 data provides an appropriate setting for this problem because it combines daily unit sales with calendar events, Supplemental Nutrition Assistance Program (SNAP) indicators, and weekly product prices. Its large size also makes it possible to demonstrate a scalable workflow rather than a small proof of concept. The notebook preserves all 27.49 million FOODS observations during data preparation and feature engineering, then uses GPU-compatible models to test detailed product-level forecasting strategies.

The work intentionally uses two levels of analysis. The detailed level reveals how difficult product-store forecasting becomes when demand is intermittent. The final business level aggregates demand to store-department series, where the signal is more stable and directly aligned with replenishment and staffing decisions. This design avoids presenting one modeling approach as universally best; instead, it selects a granularity and model that match the operational question.

2.1 Practical Context

- Inventory planners need an estimate of how many units will be demanded in each store and department over the next four weeks.
- Store managers need to anticipate weekend peaks and differences between locations.
- Supply teams need forecasts that account for product availability, price behavior, event dates, and SNAP activity.
- Data teams need a pipeline that can process tens of millions of rows without discarding the detailed history.

3. Problem Statement

The target is a non-negative count of units sold, so the project is a regression and time-series forecasting task rather than classification. At the detailed stage, one observation represents a product-store-day and the target is units sold for that product in that store on that day. At the final stage, one observation represents total daily food demand for a store-department combination.

> **Formal problem statement**
> Given historical sales, product and store identifiers, calendar timing, event indicators, SNAP status, price information, and lagged demand features, predict future daily food units sold for each store-department series over a 28-day horizon.

3.1 Modeling Questions

1. Can GPU-accelerated regression models improve on simple historical assumptions for sparse product-level demand?

2. Which model provides the best balance of validation accuracy, training speed, inference speed, and GPU resource use?

3. Does aggregating detailed demand into store-department series create a forecast that is more stable and useful for business planning?

4. Can a tuned Prophet configuration outperform a recent weekday baseline on a fully unseen final period?

3.2 Unit of Prediction

Stage	Observation	Target	Horizon
Detailed experiments	Product x store x day	Daily units sold	28 validation days, then 28 test days
Final forecast	Store x department x day	Daily department demand	28 validation days, then 28 test days

4. Objectives

1. Construct a reproducible large-scale dataset with more than several million observations while preserving sales, calendar, event, SNAP, and price information.
2. Use RAPIDS cuDF and CuPy for GPU-based loading, reshaping, joining, preprocessing, feature engineering, and metric computation.
3. Train and compare at least three cuML regression algorithms under the same chronological validation design.
4. Record GPU utilization, peak memory, execution time, training throughput, and inference throughput for the detailed models.
5. Select a final forecasting approach using validation data only, then evaluate it once on an unseen 28-day test period.
6. Translate model performance into business terms that support inventory, staffing, and supply-planning decisions.

4.1 Scope Boundaries

- The analysis is limited to the FOODS category in the M5 sales history.
- The notebook predicts observed sales, which may differ from unconstrained demand when inventory stockouts occur.
- The final forecast is produced for 30 store-department series rather than every individual product.
- The project does not claim causal effects for events, SNAP, or prices; those variables are used as predictive context.
- No CPU-versus-GPU benchmark is reported. GPU value is demonstrated through large-scale processing, throughput, and within-GPU model comparisons.

5. Data

Source, structure, quality checks, exploratory findings, preprocessing, and chronological splitting.

5.1 Data Source

The project uses the public Walmart M5 Forecasting - Accuracy competition data [1]. The source provides hierarchical unit-sales history, a calendar table, and weekly store-product prices. The notebook reads three files from Google Drive in Colab and filters the sales hierarchy to the FOODS category. The report does not redistribute the source data; it documents the transformations required to reproduce the analytical table.

Dataset page: Walmart M5 Forecasting - Accuracy

5.2 Source Tables and Analytical Scale

Table	Shape	Role
sales_train_validation.csv	30,490 x 1,919	Hierarchical product identifiers and 1,913 daily sales columns
calendar.csv	1,969 x 14	Dates, week identifiers, weekdays, events, and state SNAP flags
sell_prices.csv	6,841,121 x 4	Weekly sell price by store and product
FOODS wide table	14,370 x 1,919	1,437 food products across 10 stores
FOODS long table	27,489,810 x 6	One row per product-store-day before calendar and price joins
Clean modeling table	27,489,810 x 20	Merged and missing-price-treated detailed dataset
Final Prophet table	57,390 x 7	Thirty store-department time series

Scale confirmation

The central analytical table contains 27,489,810 rows. Every source sales row is preserved through the calendar and price merges, and row counts are checked before and after joining.

5.3 Data Preparation and Preprocessing

The source sales table is stored in wide form, with one column for every day. The notebook applies cuDF melt to convert it to long form. Identifier columns remain fixed while the daily columns become a numeric day identifier and a units-sold value. This transformation creates one row per product, store, and day, which is the structure required for time-aware joins, feature engineering, and regression.

Operation	Implementation and rationale
Filter	Keep only cat_id = FOODS, yielding 14,370 product-store series.
Encode	Convert product, department, store, state, and event labels to compact integer codes; save lookup tables for readability.
Reshape	Melt 1,913 daily columns into 27,489,810 long-format rows.
Prepare calendar	Convert dates, weekdays, months, years, events, and SNAP flags to compact types.
Prepare prices	Encode product and store keys, retain the weekly price, and verify uniqueness.
Merge	Left-join sales to calendar by day and prices by product, store, and week.
Treat missing price	Fill unavailable prices with zero and preserve a separate availability indicator.

Operation	Implementation and rationale
Optimize memory	Use int8, int16, int32, and float32 where their value ranges are sufficient.

5.4 Data Quality Findings

- Sales missing values: 0.
- Duplicate product-store series: 0.
- Zero-sales observations: 62.02%, confirming intermittent demand at the detailed level.
- Missing prices after the merge: 6,093,857 rows, or 22.17% of the analytical table.
- Missing-price rows with positive sales: 0, supporting the interpretation that the product was not yet available or not priced in those periods.
- Missing calendar dates after joining: 0; total rows before and after merging are identical.

Missing-price treatment

No sales records were deleted. sale_price was filled with zero only after confirming that every missing-price row had zero units sold, and price_is_available was added so zero price can be distinguished from a valid observed value.

5.5 Feature Engineering

Features were designed to represent identity, calendar timing, external retail context, recent price behavior, and demand history. Historical demand features use lagged values only, so the current target is not included in its own predictors. Rolling statistics are computed from units_lag_1, which shifts the series before aggregation and protects the split from direct target leakage.

Feature group	Examples	Purpose
Identifiers	product_code, department_code, store_code, state_code	Allow shared models to learn systematic differences across the hierarchy.
Calendar	day_id, day_of_week, weekend, month, year	Capture trend, weekly seasonality, and annual movement.
Events and SNAP	primary and secondary event codes/types; state SNAP flags	Represent event timing and assistance-program availability.
Price	current price, availability, 7-day lag, difference, and rate	Describe price level, product availability, and recent price movement.
Demand lags	1, 7, 14, and 28 days	Represent recent demand and repeated weekly patterns.
Rolling demand	7-, 14-, and 28-day means; 28-day standard deviation	Summarize local level and variability.
Trend	7-day mean minus 28-day mean	Indicate whether recent demand is above or below its longer-term level.

5.6 Chronological Train, Validation, and Test Split

Random splitting would allow future observations from the same product-store series to appear in training while earlier observations appear in validation, producing optimistic results. The notebook therefore keeps time order: older observations train the models, the following 28 days guide model selection, and the last 28 days provide an unbiased final evaluation. The first 28 days are excluded from detailed modeling because complete lag history is not available.

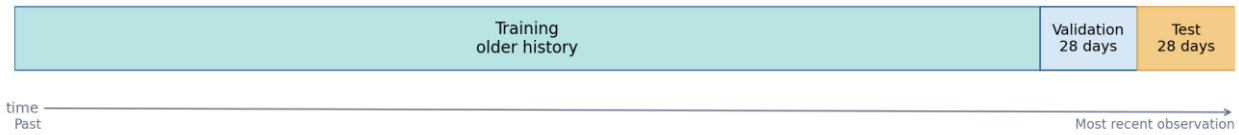


Figure 2. Chronological data split used to prevent future leakage.

Partition	Period	Size
Detailed training	Days 29-1857	26,282,730 rows before deterministic sampling
Detailed validation	Days 1858-1885	402,360 rows
Detailed test	Days 1886-1913	402,360 rows
Prophet training	Through 2016-02-28	All earlier observations for each series
Prophet validation	2016-02-29 to 2016-03-27	840 rows = 30 series x 28 days
Prophet test	2016-03-28 to 2016-04-24	840 unseen rows

5.6.1 Why the Split Sizes Differ

The detailed split counts product-store-day observations, so every product sold in every store contributes a separate row. The Prophet split is created after aggregation and therefore contains only 30 store-department series; each 28-day validation or test window contains 840 observations (30 x 28).

Split principle

Both stages follow the same chronological rule: validation data supports model selection, while the final test period remains unseen until the selected model is evaluated.

5.7 Exploratory Data Analysis

Exploratory analysis was performed on aggregated views of the full detailed table. The objective was not only to describe the data, but also to identify signals that should be represented in model design: long-term trend, hierarchy differences, weekly seasonality, event timing, and SNAP availability.

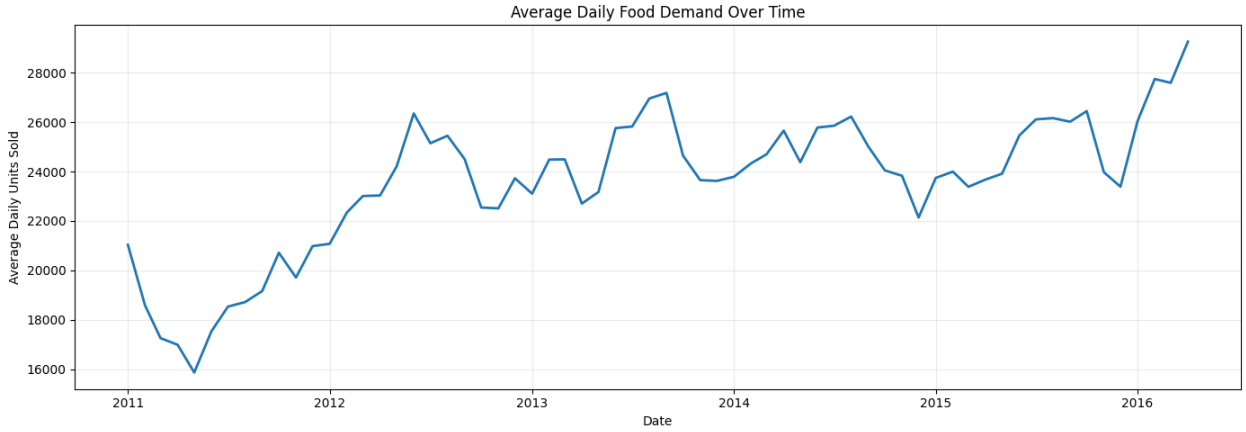


Figure 3. Average daily food demand by month, 2011-2016.

Demand falls during the early 2011 period and then rises over the longer horizon, with repeated local peaks and declines. The final months reach the highest values in the series. This pattern supports including trend and seasonal components instead of assuming a constant mean.

5.7.1 Department and Store Heterogeneity

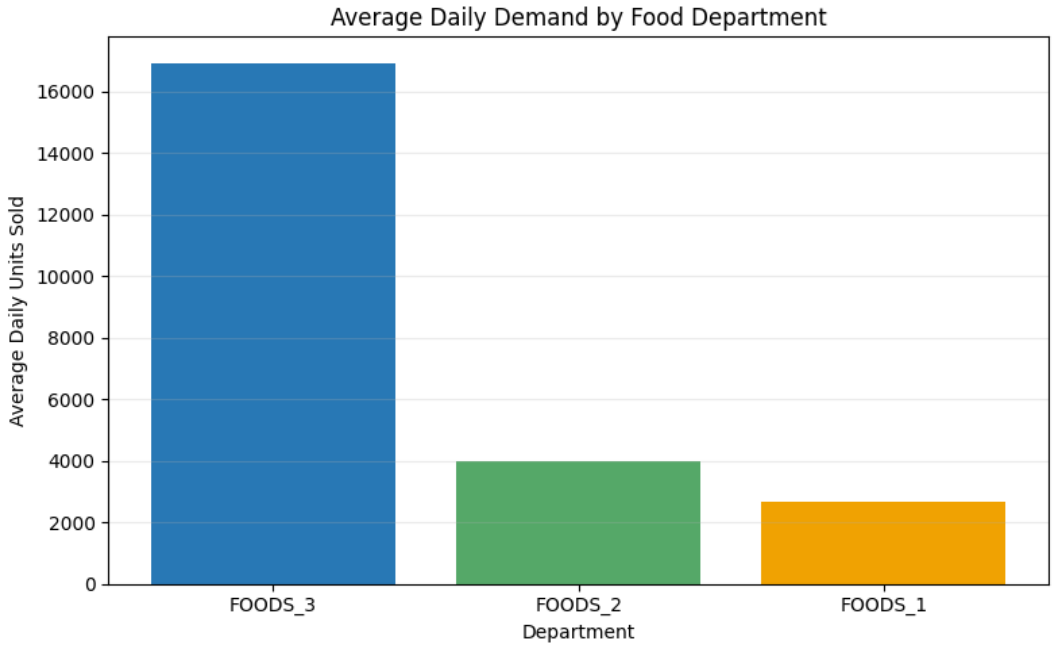


Figure 4. Average daily demand across the three food departments.

FOODS_3 averages approximately 16,922 units per day, compared with 3,988 for FOODS_2 and 2,660 for FOODS_1. The scale difference means that department identity is operationally important and that total errors should be interpreted relative to demand volume.

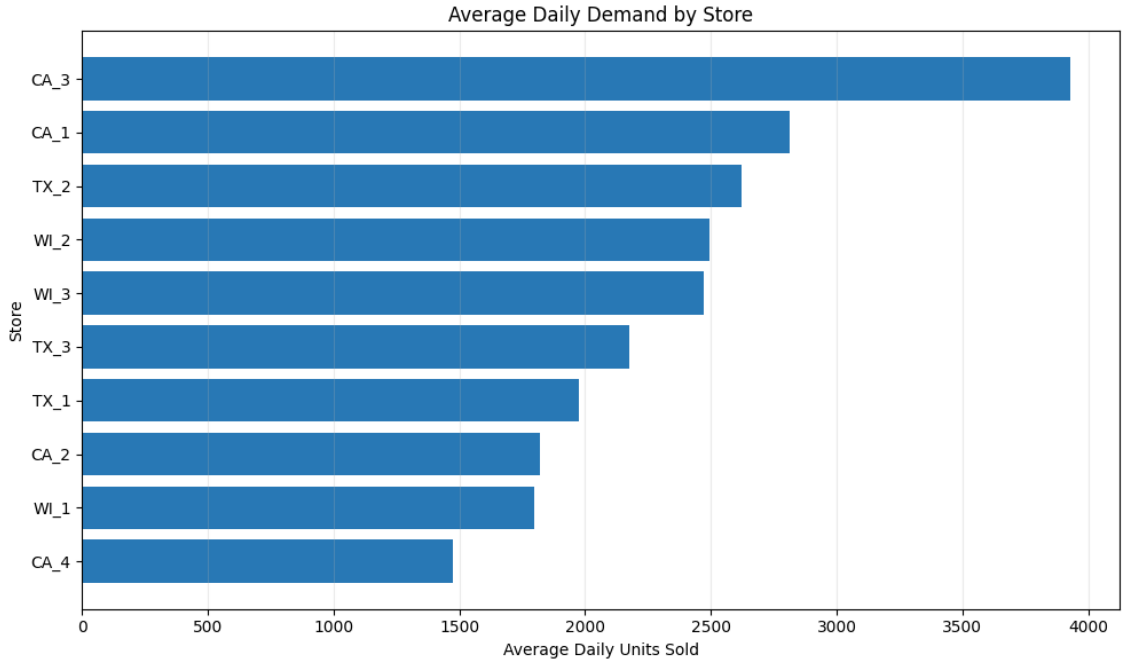


Figure 5. Average daily demand by store.

CA_3 is the highest-demand store at about 3,929 daily units, while CA_4 is the lowest at about 1,474. The spread across stores supports separate store-department time series in the final model rather than one undifferentiated national forecast.

5.7.2 Weekly, Event, and SNAP Patterns

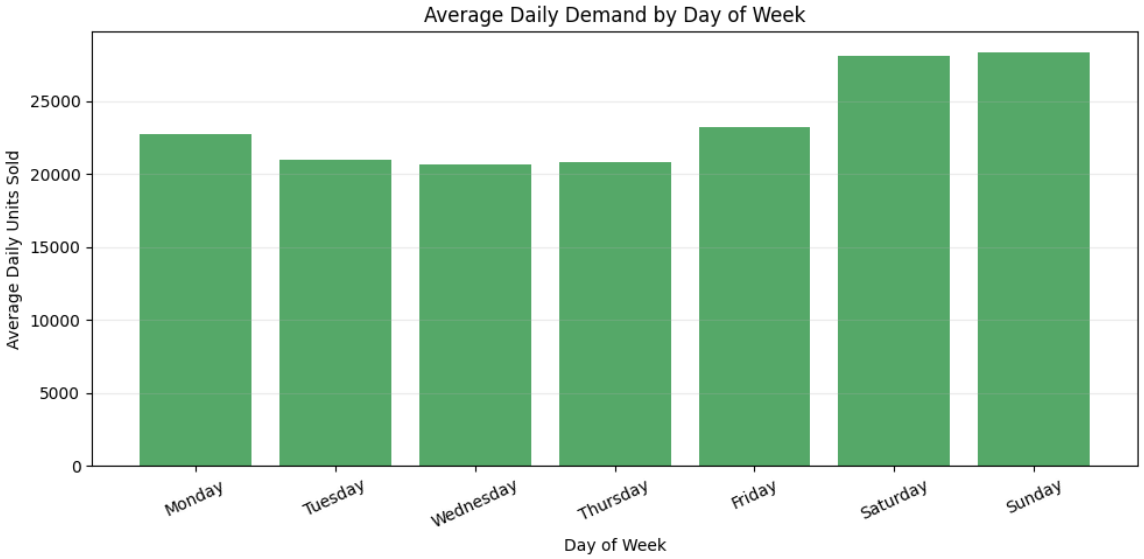


Figure 6. Average daily food demand by day of week.

Weekend demand is clearly higher: Sunday averages about 28,347 units and Saturday about 28,077, while Wednesday is lowest near 20,700. This result directly motivates the weekday baseline and weekly seasonality in Prophet.

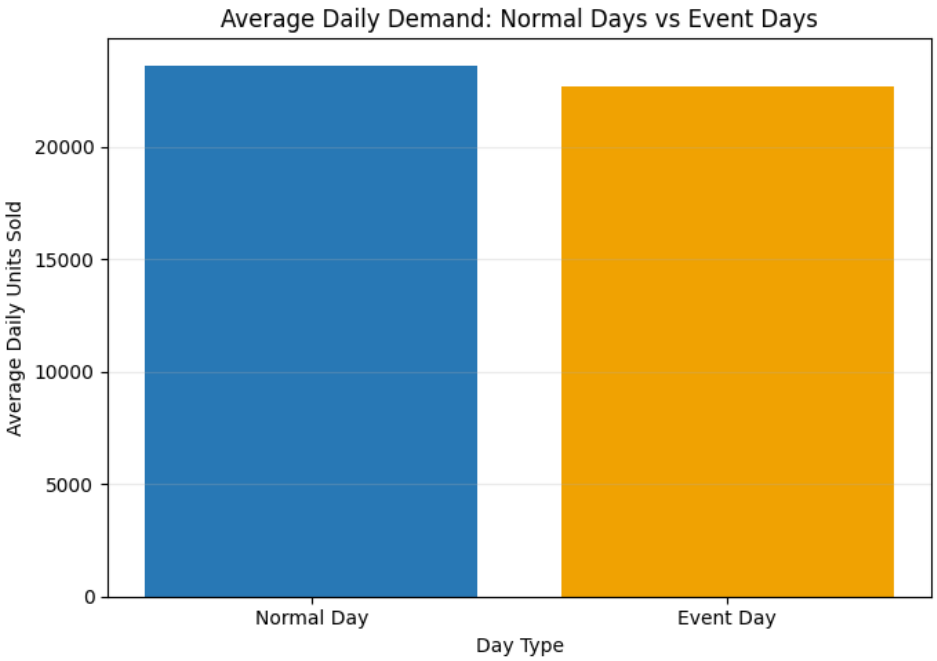


Figure 7. Average demand on normal days and event days.

Aggregated event-day demand is 3.93% below normal-day demand. This does not imply that every event reduces demand; event effects can differ in sign and magnitude. For that reason, the final model retains individual event dates rather than a single event-day flag.

5.7.3 SNAP Availability

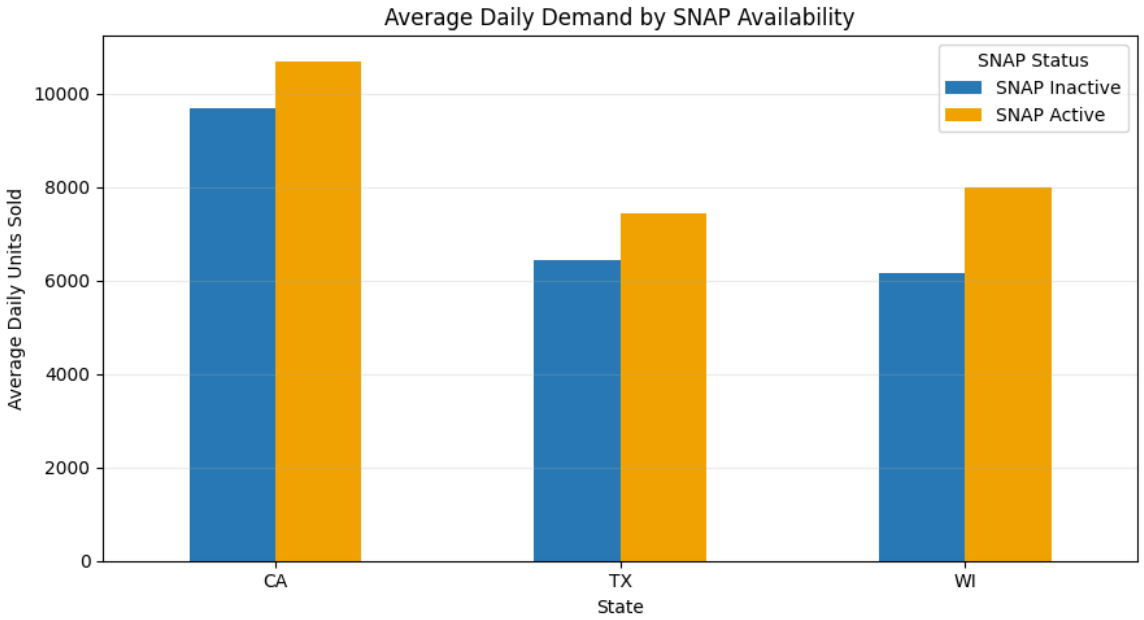


Figure 8. Average demand by SNAP status and state.

Average demand is higher on SNAP-active days in California, Texas, and Wisconsin. The increase is largest in Wisconsin, where average daily units rise from about 6,156 to 7,998. SNAP is therefore retained as an external regressor in the final forecasting model. This is a predictive association, not a causal estimate.

EDA synthesis

The visual evidence supports four design choices: hierarchy-aware modeling, weekly seasonality, flexible long-term trend, and external context from event dates and SNAP status.

6. Methodology and Training

Two modeling stages were evaluated under consistent chronological splits.

6.1 Modeling Strategy

The detailed product stage is an engineering and model-comparison stage. It tests whether GPU regression can extract useful signal from sparse product-store histories while recording resource behavior. The final store-department stage is the business-forecasting stage. It reduces intermittency and uses a decomposable time-series model that can express trend, seasonality, events, and known regressors.

Method	Core idea	Reason for inclusion
Historical baselines	Previous day, previous week, and 7-day rolling mean	Reference assumptions that require no fitted model
cuML Linear Regression	Linear additive relationship	Fast benchmark across 30 numerical features
cuML Ridge Regression	Linear model with L2 regularization	Controls instability from correlated lag and rolling features
cuML Random Forest	Nonlinear bagged trees	Captures interactions and thresholds without a linear assumption
GPU XGBoost Poisson	Boosted trees with count objective	Matches non-negative count-like sales behavior
GPU XGBoost L1	Boosted trees with absolute-error objective	Robust to spikes and aligned with MAE
4-week weekday baseline	Average of the same weekday in the latest four weeks	Strong seasonal reference for the final 28-day forecast
Original Prophet	Default-style multiplicative time-series model	Tests an untuned decomposable forecast
Tuned Prophet	Additive trend-seasonality-regressor model	Final candidate adapted to the observed retail patterns

6.2 Training Setup

Element	Implementation
Environment	Google Colab with NVIDIA Tesla T4 GPU and 15,360 MB memory
GPU libraries	cuDF 26.02.01, CuPy, cuML, XGBoost 3.3.0 with CUDA histogram training
Monitoring	NVML utilization and memory sampled every 0.20 seconds; CUDA synchronization around timed operations
Detailed training matrix	2,021,749 deterministic training rows x 30 features

Element	Implementation
Detailed validation	All 402,360 rows from the next 28 days
Final time series	30 independent store-department series with 1,913 observations each
Model selection	Lowest validation WMAPE, with MAE, RMSE, speed, and resource use as supporting evidence

6.3 Evaluation Metrics

Because demand is numeric and non-negative, evaluation focuses on regression error. Predictions are clipped at zero before metrics are computed. WMAPE is the primary selection metric because the detailed data contains many zeros and because a weighted percentage communicates the total business error more clearly than an error measured only in raw units.

Metric	Definition	Business meaning	Direction
MAE	mean(|actual - forecast|)	Typical absolute miss in units; easy to explain operationally	Lower is better
RMSE	sqrt(mean((actual - forecast)^2))	Penalizes large misses that can create serious stockouts or excess inventory	Lower is better
R-squared	1 - residual variation / total variation	Share of aggregated demand variation explained by the forecast	Higher is better
WMAPE	sum(|actual - forecast|) / sum(actual) x 100	Total absolute error as a percentage of total demand; stable with zero individual observations	Lower is better
Weighted forecast accuracy	100 - WMAPE	Intuitive restatement of proportional accuracy; not classification accuracy	Higher is better

Why WMAPE instead of MAPE

MAPE divides by each individual actual value and becomes undefined or unstable when actual sales are zero. WMAPE divides total absolute error by total demand, so all rows can be retained and higher-volume demand receives appropriate weight.

6.4 Hyperparameter Tuning Approach

The notebook uses controlled, hand-specified experiments rather than an automated grid or Bayesian search. The base and tuned Random Forest configurations isolate the effect of more trees, deeper trees, additional bins, a larger feature fraction, and a larger row sample. XGBoost uses early stopping on the chronological validation set. Prophet compares an original configuration with a tuned configuration informed by EDA and validation results. This approach keeps the computational story transparent and reproducible.

6.5 Detailed Product-Demand Experiments

6.5.1 Baselines and cuML Models

The three historical baselines test whether tomorrow resembles yesterday, the same weekday one week earlier, or the average of the last seven days. The 7-day rolling mean is strongest because averaging reduces the effect of isolated zero days and spikes. It establishes a WMAPE threshold of 65.25% that fitted models should improve.

Method	MAE	RMSE	WMAPE (%)
7-Day Rolling Mean	1.2827	2.6864	65.2548
Previous Day	1.5137	3.1850	77.0064
Previous Week	1.5645	3.3978	79.5890
cuML Linear Regression	1.2694	2.5595	64.5773
cuML Ridge Regression	1.2703	2.5597	64.6252
cuML Base Random Forest	1.2670	2.6343	64.4548
cuML Tuned Random Forest	1.2636	2.5679	64.2840

Linear, Ridge, and Random Forest models all improve modestly on the rolling baseline. Ridge trains in only 0.15 seconds, while the tuned Random Forest requires 48.29 seconds for a relatively small WMAPE improvement over the base forest. This is the first clear complexity tradeoff: additional model capacity does not automatically produce proportional forecasting value.

6.5.2 GPU XGBoost Objectives

Poisson loss is designed for count-like targets and enforces a non-negative mean structure. Absolute-error loss (L1) minimizes the median-oriented absolute deviation and is less sensitive than squared error to occasional sales spikes. Both models use CUDA histogram training, 10-level trees, row and feature subsampling, L1/L2 regularization, and early stopping.

Model	Trees	Best iteration	MAE	RMSE	WMAPE	Train time
XGBoost Poisson	500 max	232	1.2226	2.4464	62.1990	6.6576 s
XGBoost L1	600 max	214	1.1485	2.5045	58.4270	9.1806 s

Detailed-stage selection

XGBoost L1 produced the lowest MAE and WMAPE. XGBoost Poisson produced the lowest RMSE, showing better control of the largest errors, but L1 delivered the stronger overall proportional accuracy.

7. Evaluation and Results

7.1 Complete Product-Level Validation Comparison

Model	MAE	RMSE	WMAPE (%)
XGBoost L1	1.1485	2.5045	58.4270
XGBoost Poisson	1.2226	2.4464	62.1990
cuML Tuned Random Forest	1.2636	2.5679	64.2840
cuML Base Random Forest	1.2670	2.6343	64.4548
cuML Linear Regression	1.2694	2.5595	64.5773
cuML Ridge Regression	1.2703	2.5597	64.6252
7-Day Rolling Mean	1.2827	2.6864	65.2548
Previous Day	1.5137	3.1850	77.0064
Previous Week	1.5645	3.3978	79.5890

XGBoost L1 reduces validation WMAPE by 6.83 percentage points relative to the strongest simple baseline, an approximately 10.5% relative reduction in proportional error. The ranking also demonstrates why several objectives were tested: the Poisson objective minimizes the reported RMSE, while the L1 objective minimizes MAE and WMAPE.

7.2 GPU Performance Analysis

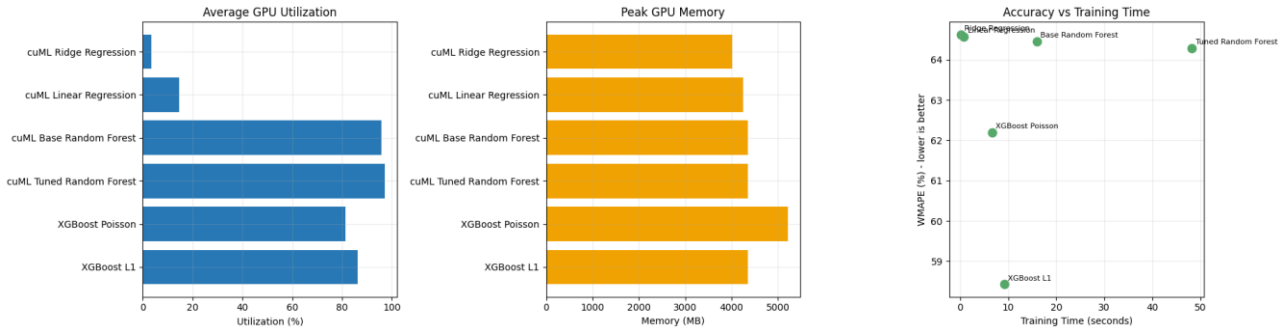


Figure 9. GPU utilization, peak memory, and WMAPE-versus-training-time comparison.

7.2.1 Resource and Complexity Findings

Model	Train s	Infer s	Avg GPU %	Peak MB	WMAPE
cuML Ridge	0.1466	0.0094	3.50	4,020	64.6252
cuML Linear	0.7842	0.1784	14.67	4,252	64.5773

Model	Train s	Infer s	Avg GPU %	Peak MB	WMAPE
cuML Base RF	15.9571	0.3381	95.79	4,354	64.4548
cuML Tuned RF	48.2913	0.6351	97.29	4,350	64.2840
XGBoost Poisson	6.6576	0.0670	81.51	5,220	62.1990
XGBoost L1	9.1806	0.0461	86.34	4,344	58.4270

- Random Forest models sustained the highest average utilization, near 96-97%, because many trees were built concurrently over the GPU training matrix.
- XGBoost Poisson used the most peak memory, about 5.22 GB, while XGBoost L1 remained near 4.34 GB and produced the best WMAPE.
- Ridge was the fastest training and inference method, but accuracy remained close to Linear Regression and the historical rolling mean.
- Tuned Random Forest tripled training time relative to the base forest for only a 0.17-point WMAPE reduction.
- XGBoost L1 represents the strongest detailed tradeoff: the best proportional accuracy, 9.18-second training, and 0.046-second validation inference.

Utilization values must be interpreted with duration. Very short jobs may finish between monitoring samples, so low average utilization does not mean that GPU kernels were not used. The synchronized training and inference times, throughput, maximum utilization, and peak memory provide a more complete view than a single post-cell snapshot.

7.3 Final Product-Level Test

Method	MAE	RMSE	WMAPE (%)
XGBoost L1	1.1322	2.3657	57.5959
7-Day Rolling Mean	1.2862	2.5630	65.4303

The selected XGBoost L1 model remains stronger on the unseen detailed test period. It reduces WMAPE by 7.83 percentage points, or about 12.0% relative to the rolling baseline. However, a 57.60% WMAPE remains too high for a stable operational product forecast, largely because zero-sales days dominate the series. This evidence motivates aggregation rather than hiding the difficulty.

7.4 Transition to the Final Business Forecast

The detailed experiment answers a technical question about sparse product demand, but business planning often occurs at a higher hierarchy. The notebook sums units by date, store, and department to create 30 continuous time series. This aggregation reduces random product-level zeros, preserves important differences between stores and departments, and produces a target that better matches inventory and staffing decisions.

Field	Type	Meaning
ds	Date	Daily timestamp required by Prophet

Field	Type	Meaning
store_code	Integer	Identifies one of 10 stores
department_code	Integer	Identifies one of 3 food departments
y	Integer	Total daily units sold for the store-department series
average_price	Float	Mean price among products with available prices
available_products	Integer	Count of products with valid price availability
snap_active	Binary	State-specific SNAP flag for the date

- Rows: 57,390.
- Series: 30 = 10 stores x 3 departments.
- History: 2011-01-29 through 2016-04-24.
- Observations per complete series: 1,913.
- Event dates: primary event codes are supplied to Prophet as named holidays on their exact dates.

Important comparison rule

Product-level WMAPE and store-department WMAPE should not be compared as if they were the same task. Aggregation changes the target scale and removes much of the intermittency; the final metric describes a different, more stable business decision level.

7.5 Final Model: Tuned Prophet

Prophet is a decomposable time-series forecasting model originally designed for business series with trend changes, multiple seasonal patterns, and event effects [2]. Instead of treating time as just another numeric column, it builds the forecast from interpretable components. In this project the point forecast can be summarized as:

Forecast structure

predicted demand = trend + weekly seasonality + yearly seasonality + monthly seasonality + event effects + regressor effects

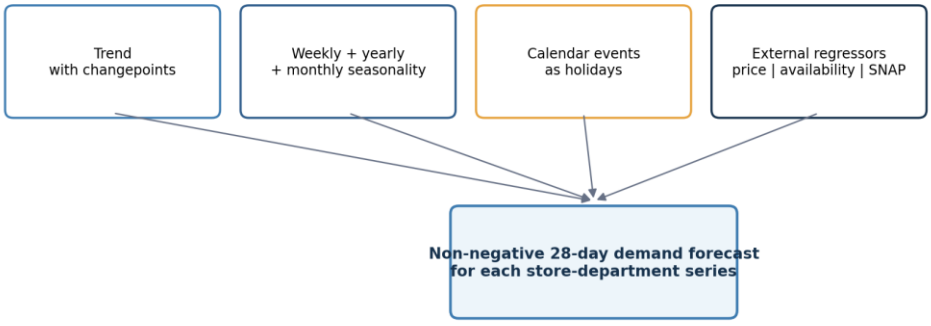


Figure 10. Components used by the tuned Prophet forecast.

7.5.1 Trend and Changepoints

The trend component estimates the underlying direction of demand and allows its slope to change at candidate changepoints. The parameter changepoint_prior_scale controls how readily those changes are accepted. The tuned value of 0.20 is more flexible than Prophet's default, which is appropriate because the EDA shows multi-year growth interrupted by local rises and declines. Greater flexibility is useful, but validation is necessary to prevent the trend from following noise.

7.5.2 Seasonal Components

Weekly seasonality captures the clear weekend peak and midweek decline. Yearly seasonality represents recurring annual movement. A custom monthly component with a period of 30.5 days and Fourier order 5 was added because the monthly aggregate series shows repeated shorter cycles that are not fully described by the weekly or yearly terms. Fourier terms create smooth periodic curves rather than a separate unconstrained parameter for every date.

7.5.3 Additive Mode, Events, and Regressors

The original model uses multiplicative seasonality, which scales seasonal effects in proportion to the trend level. The tuned model uses additive seasonality, so recurring effects are added in units. Validation showed that the additive configuration better matched these aggregated series. The model also includes the exact dates of individual primary events as holidays. This is more appropriate than assuming one universal event effect, because the EDA found that event days as a group were slightly lower than normal days.

Regressor	Definition	Forecasting role
average_price	Average price among available products	Allows the forecast to respond to the price environment
available_products	Number of products with valid prices	Distinguishes a low-demand day from a day with fewer available products
snap_active	State-specific SNAP availability	Represents the higher-demand pattern observed on SNAP-active days

7.5.4 Tuned Configuration

Parameter	Value	Purpose
seasonality_mode	additive	Seasonal and event effects are added in demand units
yearly_seasonality	True	Models annual repetition
weekly_seasonality	True	Models strong weekday/weekend differences
daily_seasonality	False	Data has one observation per day, so within-day cycles are unavailable
monthly seasonality	period 30.5, Fourier order 5	Adds a smooth monthly pattern

Parameter	Value	Purpose
changepoint_prior_scale	0.20	Allows more responsive trend changes
seasonality_prior_scale	10	Controls seasonal flexibility
holidays_prior_scale	10	Controls event-effect flexibility
training history	Latest 1,095 days	Focuses the fit on the most recent three years
uncertainty_samples	0	Produces fast point forecasts; prediction intervals are not estimated

7.5.5 Series-Wise Training Procedure

The final solution does not fit one global Prophet model. It loops over all 30 store-department combinations and fits one independent model to each series. This allows each store and department to have its own trend, seasonal amplitude, event response, and regressor coefficients. The results are then concatenated into one evaluation table.

1. Sort each store-department series by date.
2. Keep observations no later than the training cutoff; for the tuned model, retain the most recent 1,095 days.
3. Build the tuned Prophet specification with holidays and three regressors.
4. Fit the model using ds, y, average_price, available_products, and snap_active.
5. Predict the exact validation or test dates using their known regressor values.
6. Clip negative predictions to zero because demand cannot be negative.
7. Combine all 30 series and calculate aggregate evaluation metrics.

Standard Prophet fitting is CPU-based; the large-scale preparation that produces the 57,390-row forecasting table is executed with RAPIDS on the GPU. The aggregated table is intentionally small enough for rapid series-wise Prophet training. The final Prophet execution time is therefore reported as wall-clock execution time rather than as a GPU training metric.

> **Deployment dependency**
> The regressors used for a future forecast must be known or planned for the forecast horizon. SNAP dates can be scheduled, while price and product availability should come from pricing and inventory plans or be forecast separately.

7.6 Final-Stage Validation Results

All three methods forecast the same 28-day validation period. The baseline uses the mean of each weekday from the previous four weeks. Original Prophet uses full history with multiplicative seasonality. Tuned Prophet uses additive seasonality, a monthly component, more flexible trend changes, external regressors, events, and the most recent three years of history.

Method	MAE	RMSE	R2	WMAPE	Accuracy	Time
Tuned Prophet	100.0650	158.3424	0.9620	10.6274	89.3726	20.06 s
4-Week Weekday Baseline	106.3976	169.5769	0.9564	11.3000	88.7000	0.14 s
Original Prophet	126.7248	209.9206	0.9332	13.4588	86.5412	26.85 s

- Tuned Prophet has the lowest MAE, RMSE, and WMAPE, and the highest R-squared and weighted forecast accuracy.
- Relative to Original Prophet, tuning reduces WMAPE by 2.83 percentage points, an approximately 21% relative reduction in proportional error.
- Relative to the strong weekday baseline, Tuned Prophet reduces WMAPE by 0.67 percentage points and RMSE by about 6.6%.
- Original Prophet is worse than the baseline, confirming that a familiar model should not be accepted without validation and that default settings were not appropriate for this demand structure.
- Model selection uses validation only; the final test remains untouched until after Tuned Prophet is selected.

Validation decision

Tuned Prophet is selected because it ranks first on the primary WMAPE metric and also leads every supporting accuracy metric. The decision is made before the final test is opened.

7.7 Final Unseen Test Results

After selection, Tuned Prophet is refitted through the end of the validation period and evaluated once on 28 unseen days from 2016-03-28 through 2016-04-24. The weekday baseline is evaluated on the same dates. This final comparison estimates generalization after all configuration decisions have already been made.

Method	MAE	RMSE	R2	WMAPE (%)	Weighted accuracy (%)
Tuned Prophet	96.9911	146.7876	0.9679	10.3008	89.6992
4-Week Weekday Baseline	98.1506	151.7855	0.9657	10.4240	89.5760

Result	Interpretation
MAE = 96.99	A typical forecast differs from actual demand by about 97 units per store-department-day.
RMSE = 146.79	Larger errors remain controlled; RMSE is only moderately above MAE despite daily peaks.
R2 = 0.9679	The predictions explain about 96.79% of variation in aggregated demand.
WMAPE = 10.30%	Total absolute error is 10.30% of total observed demand across the test set.
Weighted accuracy = 89.70%	An intuitive restatement of WMAPE; it is not classification accuracy.
Execution = 19.76 s	All 30 series were fitted, predicted, and evaluated in under 20 seconds.

The final test is slightly better than validation rather than materially worse: WMAPE improves from 10.63% to 10.30% and R-squared rises from 0.9620 to 0.9679. This close performance supports good generalization and provides no evidence of overfitting from the validation-to-test comparison. The conclusion is appropriately limited to the available 28-day horizon.

7.8 Forecast Behavior and Error Analysis

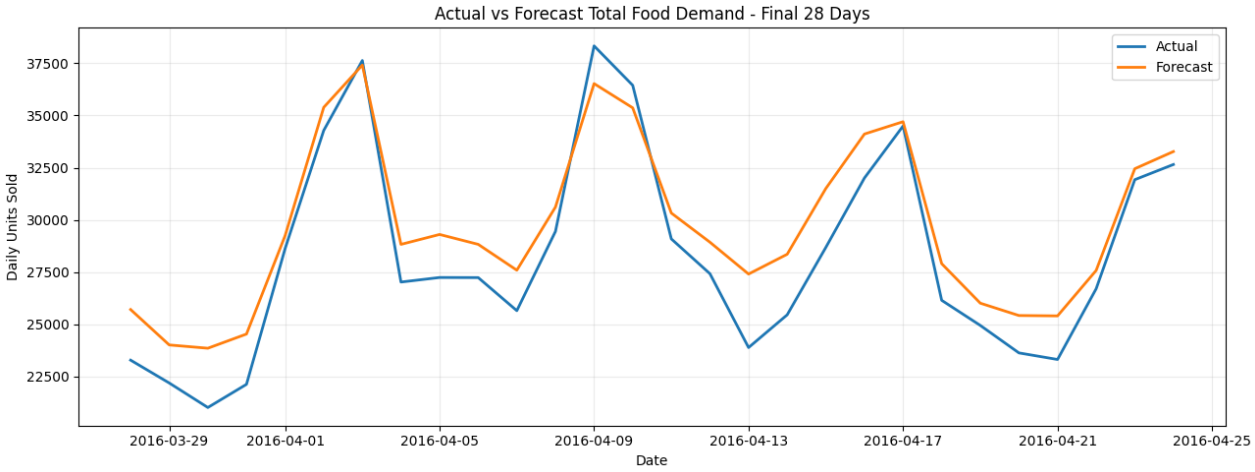


Figure 11. Actual and forecast total food demand across the final 28 days.

The forecast follows the weekly rise-and-fall pattern and captures the timing of major peaks around early and mid-April. It slightly overestimates several troughs and underestimates the highest peak around 9 April. This pattern is consistent with a smooth decomposable model: it is strong at recurring structure but can damp isolated extremes.

7.8.1 Where the Final Model Can Fail

- Sudden one-day demand spikes can exceed the smooth seasonal and trend components.
- Unexpected stockouts or supply constraints may cause observed sales to understate true customer demand.
- Future price and availability assumptions can be wrong, introducing regressor error.
- New stores, departments, or structural changes outside the training history are not covered by the fitted series.
- The final model returns point forecasts because uncertainty sampling was disabled; prediction intervals are not available in this notebook.

7.8.2 Feature Interpretation

The final model is partially interpretable by construction: trend, seasonal patterns, holidays, and external regressors have distinct roles. The EDA supports the inclusion of weekly seasonality and SNAP, while event dates are retained individually because their aggregate effect is not uniformly positive. The notebook does not run SHAP, permutation importance, or a ranked coefficient analysis, so no empirical feature-importance ranking is claimed. A future interpretability extension should report component plots and regressor coefficients for each store-department model.

8. Challenges

Challenge	Observed issue	Resolution
Large intermediate table	Multiple 27.5-million-row matrices can exceed Colab T4 memory.	All rows were processed with RAPIDS; detailed model fitting used a deterministic 2.02-million-row training sample while validation and test remained complete.
Sparse demand	62.02% of product-store-day targets are zero.	Historical baselines, Poisson loss, and L1 loss were tested; the final business target was aggregated to reduce intermittency.
Missing weekly prices	22.17% of merged rows lack a price before product availability.	Positive-sales checks confirmed the pattern; price was set to zero and a separate availability indicator was retained.
Leakage risk	Random splits and unshifted rolling windows would reveal future demand.	Chronological partitions, lagged sales, and rolling statistics based on the previous day protected the evaluation.
Model-selection bias	Repeated access to the final period would invalidate the test.	Tuning and selection used validation only; the 28-day test was evaluated once after selection.
Mixed computation stages	RAPIDS models operate on GPU while standard Prophet fits on CPU.	GPU work was monitored separately; Prophet was applied only after GPU aggregation reduced the table to 57,390 rows.

8.1 Lessons from the Challenges

Scalability is not only a question of whether one operation runs on a GPU. It depends on controlling data types, deleting obsolete tables, releasing CuPy memory blocks, synchronizing timed work, and avoiding simultaneous copies of large matrices. Likewise, forecasting quality is not solved only by increasing model complexity. Granularity, target sparsity, and time-aware evaluation had a larger effect on business usefulness than the difference between several detailed regressors.

9. Future Work

- Train the detailed XGBoost model on all 26.28 million training rows using a higher-memory GPU or distributed RAPIDS with Dask.
- Add inventory levels and stockout indicators so the target can be separated into unconstrained demand and supply-limited sales.
- Include promotion flags, markdowns, local weather, supplier lead times, holidays by market, and store operating changes.
- Generate probabilistic prediction intervals and convert them into safety-stock recommendations with service-level targets.
- Run rolling-origin cross-validation across several 28-day windows to measure stability across seasons rather than relying on one validation and one test window.
- Compare Tuned Prophet with modern global forecasting models after defining a resource budget and preserving the same chronological evaluation.

- Report store-department error distributions and component plots to identify series that require custom models or manual overrides.
- Deploy scheduled monitoring for WMAPE, bias, stockout cost, and forecast drift; retrain when thresholds are exceeded.

9.1 Recommended Deployment Path

1. Automate daily ingestion and repeat the cuDF preprocessing pipeline.
2. Validate known future regressors for the next 28 days.
3. Fit or load the 30 Tuned Prophet models and create non-negative forecasts.
4. Map store and department codes back to readable identifiers.
5. Publish forecasts with actuals, confidence information, and exception flags.
6. Track error after actual sales arrive and trigger retraining or review when error increases.

10. Discussion

10.1 Meaning of the Results

The project demonstrates two complementary outcomes. First, large-scale detailed data can be transformed and modeled efficiently on a Colab T4 using RAPIDS when memory is managed deliberately. Second, the most useful operational forecast is not necessarily the most detailed forecast. The move from product-store-day targets to store-department-day targets converts an intermittent signal into a stable planning series and enables a final WMAPE near 10%.

Tuned Prophet succeeds because its structure matches the data revealed by EDA. Weekly seasonality reflects weekend peaks, a flexible trend follows multi-year movement, the monthly term captures medium-cycle variation, event dates are modeled separately, and external regressors add commercial context. Its improvement over the weekday baseline is modest on the final test, but it is consistent across MAE, RMSE, R-squared, and WMAPE. That consistency is more credible than a large improvement on only one metric.

10.2 Practical Value

- A 28-day store-department forecast can guide department-level purchasing and labor planning.
- The hierarchy makes it possible to allocate expected demand across stores with very different historical volumes.
- The final 19.76-second execution makes frequent retraining or scenario refreshes practical at this scale.
- Saved Prophet models, readable predictions, comparison tables, and GPU performance files support auditability and downstream reporting.

10.3 Limitations and Responsible Use

The results describe one historical retailer dataset and should not be assumed to transfer unchanged to other regions, categories, or periods. Observed sales are not guaranteed to equal unconstrained demand, particularly during stockouts. Event and SNAP variables are predictive context, not evidence of causal impact. Store-department aggregation improves stability but cannot identify which product within a department should be replenished.

Operational decisions should consider asymmetric costs. Underforecasting can produce stockouts and service failure, while overforecasting can increase waste and holding cost. The reported metrics treat

positive and negative absolute errors symmetrically. A production system should add bias monitoring, business-cost weighting, human review for exceptional events, and uncertainty intervals before automating purchasing decisions.

11. Conclusion

This project built an end-to-end large-scale demand-forecasting workflow from the Walmart M5 sales, calendar, and price data. RAPIDS processed 27,489,810 food product-store-day observations, created a leakage-safe feature set, trained three cuML regression algorithms, evaluated two GPU XGBoost objectives, and collected utilization, memory, time, and throughput metrics. XGBoost L1 was the strongest detailed model, but its 57.60% test WMAPE also quantified the difficulty of sparse product-level demand.

The final forecasting stage aggregated demand into 30 store-department series and compared a weekday baseline with Original Prophet and Tuned Prophet. The tuned configuration was selected on validation only, then achieved 96.99 MAE, 146.79 RMSE, 0.9679 R-squared, 10.30% WMAPE, and 89.70% weighted forecast accuracy on the unseen final 28 days. The actual-versus-forecast curve confirms that the model follows the main weekly pattern and most peaks, while error analysis identifies remaining weaknesses at sharp extremes.

11.1 Metric Selection Rationale

WMAPE was used as the primary metric because it expresses total absolute error as a percentage of total actual demand and remains usable when individual observations are zero. This is essential for a dataset in which 62.02% of detailed sales observations are zero. MAE and RMSE provide unit-based context, while R-squared was added at the aggregated Prophet stage because the store-department series are stable enough for explained variation to be meaningful. R-squared was not emphasized in the sparse product stage, where proportional and absolute forecast errors are more directly actionable.

Final takeaway

The strongest result is not only the 89.70% weighted forecast accuracy. It is the disciplined chain of evidence: full-scale GPU preparation, fair chronological model comparison, validation-only selection, and confirmation on a final unseen period.

12. References

[1] Walmart. M5 Forecasting - Accuracy. Kaggle competition dataset. https://www.kaggle.com/competitions/m5-forecasting-accuracy/data

[2] Taylor, S. J., & Letham, B. (2018). Forecasting at Scale. The American Statistician, 72(1), 37-45. https://doi.org/10.1080/00031305.2017.1380080

[3] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). The M5 accuracy competition: Results, findings, and conclusions. International Journal of Forecasting, 38(4), 1346-1364. https://doi.org/10.1016/j.ijforecast.2021.11.013

[4] NVIDIA. RAPIDS cuDF documentation. https://docs.rapids.ai/api/cudf/stable/

[5] NVIDIA. RAPIDS cuML documentation. https://docs.rapids.ai/api/cuml/stable/

[6] XGBoost developers. XGBoost documentation. https://xgboost.readthedocs.io/

[7] Prophet developers. Prophet documentation. https://facebook.github.io/prophet/

[8] NVIDIA. NVIDIA Management Library (NVML) documentation.
https://developer.nvidia.com/management-library-nvml

Referenced Software Versions Recorded by the Notebook

- cuDF 26.02.01.
- XGBoost 3.3.0.
- CUDA 13.0 reported by nvidia-smi.
- NVIDIA Tesla T4 with 15,360 MB GPU memory.
- Prophet and scikit-learn were used for final time-series forecasting and aggregated metrics; exact package versions were not printed by the notebook.

Appendix A. Detailed Feature Dictionary

Feature	Type	Description
product_code	int16 / float32 matrix	Encoded product identifier
department_code	int8 / float32 matrix	Encoded food department
store_code	int8 / float32 matrix	Encoded store
state_code	int8 / float32 matrix	Encoded state
day_id	int16	Sequential day number
day_of_week	int8	Monday=0 through Sunday=6
is_weekend	int8	Saturday or Sunday indicator
month	int8	Calendar month
year	int16	Calendar year
primary_event_code	int8	Encoded first event name
primary_event_type_code	int8	Encoded first event type
secondary_event_code	int8	Encoded second event name
secondary_event_type_code	int8	Encoded second event type
snap_CA	int8	California SNAP availability
snap_TX	int8	Texas SNAP availability
snap_WI	int8	Wisconsin SNAP availability
sale_price	float32	Weekly product-store price, zero if unavailable

Feature	Type	Description
price_is_available	int8	Original price observed indicator
price_lag_7	float32	Price seven days earlier
price_difference_7	float32	Current price minus seven-day lag
price_change_rate_7	float32	Relative seven-day price change
units_lag_1	int16	Units sold one day earlier
units_lag_7	int16	Units sold seven days earlier
units_lag_14	int16	Units sold fourteen days earlier
units_lag_28	int16	Units sold twenty-eight days earlier
units_rolling_mean_7	float32	Mean of lagged demand over seven days
units_rolling_mean_14	float32	Mean of lagged demand over fourteen days
units_rolling_mean_28	float32	Mean of lagged demand over twenty-eight days
units_rolling_std_28	float32	Standard deviation of lagged demand over twenty-eight days
demand_trend_7_vs_28	float32	Seven-day mean minus twenty-eight-day mean

Appendix B. Model Configuration Summary

Model	Configuration
cuML Linear Regression	Default LinearRegression
cuML Ridge	alpha=10.0
Base Random Forest	50 trees; max_depth=12; max_features=0.8; min_split=10; min_leaf=5; n_bins=64; max_samples=0.8; n_streams=2
Tuned Random Forest	80 trees; max_depth=16; max_features=0.9; min_split=8; min_leaf=3; n_bins=128; max_samples=0.9; n_streams=1
XGBoost Poisson	500 max trees; depth=10; learning_rate=0.07; row sample=0.8; feature sample=0.9; alpha=0.1; lambda=5; early stopping=30
XGBoost L1	600 max trees; depth=10; learning_rate=0.06; row sample=0.8; feature sample=0.9; alpha=0.1; lambda=5; early stopping=30

Model	Configuration
Original Prophet	Multiplicative seasonality; yearly and weekly terms; full history; events; price, availability, and SNAP regressors
Tuned Prophet	Additive seasonality; yearly, weekly, and monthly terms; changepoint prior=0.20; latest 1,095 days; events and regressors

Appendix C. Saved Project Outputs

Output	Purpose
daily_demand_xgboost_l1.ubj	Selected detailed XGBoost model
prophet_models/*.json	Thirty fitted final Prophet models
prophet_test_predictions.parquet	Readable actual and predicted final test records
prophet_validation_comparison.csv	Validation results for final-stage candidates
prophet_final_comparison.csv	Unseen final test comparison
gpu_model_performance.csv	Training, inference, utilization, memory, and throughput
product_validation_comparison.csv	Detailed validation rankings
product_test_comparison.csv	Detailed final test comparison

Appendix D. Reproducibility Notes

- Run the notebook from the environment setup cell so RAPIDS and monitoring utilities are initialized before GPU work.
- Keep the three M5 files under the configured data directory and preserve their original filenames.
- Use the deterministic training-sample expression and random_state=42 values shown in the notebook.
- Do not randomize time-series rows; all validation and test cutoffs are date based.
- Record actual library versions and GPU hardware when rerunning, because execution time can change across environments.
- Treat the final test as a one-time evaluation after configuration choices are complete.